

Lifecycle-Aware State Spilling (LASS) for Mitigating the Latency Cliff in Managed Blockchain Clients

Abstract

Predictable execution latency is critical for blockchain validators to meet consensus deadlines. However, managed blockchain clients often experience severe execution jitter caused by the interaction between complex in-memory state management and reactive garbage collection (GC) heuristics. This paper identifies a performance instability termed the Latency Cliff, where excessive object allocation during state-trie updates overwhelms the runtime's reclamation capacity, resulting in large Stop-The-World (STW) pauses and amplified tail latency.

To address this problem, we propose Lifecycle-Aware State Spilling (LASS), a lightweight control architecture that proactively manages the lifecycle of ephemeral state objects. Rather than relying on reactive garbage collection, LASS monitors runtime memory signals and preemptively materializes transient state data to persistent storage before promotion pressure escalates. This approach effectively decouples state persistence from the unpredictable behavior of managed runtimes.

We evaluate LASS using two major enterprise blockchain clients across different managed runtimes: the JVM-based Hyperledger Besu and the CLR-based Nethermind. Experimental results show that LASS significantly stabilizes execution behavior. In the Besu environment, LASS reduces total GC time by 96.2% and decreases P99 Stop-The-World pause durations by 35.4%. In Nethermind, proactive stabilization of the Large Object Heap (LOH) yields a 5.7% improvement in sustained transaction throughput while reducing execution tail latency.

These results demonstrate that application-aware memory lifecycle management can substantially improve execution predictability in managed blockchain clients without requiring modifications to the underlying runtime systems.

1. Introduction: The Predictability Challenge in Managed Clients

The stability and performance of validator nodes in decentralized networks depend not only on deterministic state transitions for reproducibility, but also on highly predictable execution latency. In practice, however, managed blockchain clients frequently exhibit non-deterministic execution jitter. Standard garbage collection (GC) cycles introduce unpredictable pauses that can severely disrupt time-sensitive Byzantine Fault Tolerance (BFT) consensus rounds. These pauses may ultimately degrade network health by causing missed slots or accidental forks.

The root of this predictability challenge lies in the interaction between cryptographic state-trie management and reactive GC heuristics. To characterize this systemic instability, we formalize the concept of the **Latency Cliff (CL)**. The Latency Cliff refers to the critical threshold at which the object allocation rate (Ra) required for materializing the world state exceeds the concurrent reclamation rate (Rr) of the underlying garbage collector. When this threshold is crossed, promotion pressure and object graph complexity overwhelm reactive reclamation mechanisms, resulting in severe performance degradation. Although Ra and Rr are difficult to measure precisely in production environments, this imbalance manifests empirically as abrupt spikes in GC pause times and severe tail-latency amplification, which we observe in our baseline experiments.

To address this fundamental disconnect, this paper introduces **Lifecycle-Aware State Spilling (LASS)**. LASS acts as a lightweight, lifecycle-aware control layer that explicitly decouples storage persistence from the unpredictable GC lifecycle. Specifically, LASS continuously monitors runtime memory signals and proactively materializes ephemeral state objects directly to the persistent backend, bypassing the managed heap whenever necessary. By shifting memory management from a reactive to a proactive model, LASS mitigates the extreme promotion pressure that triggers the Latency Cliff.

More broadly, the Latency Cliff illustrates how allocation-heavy blockchain workloads can interact poorly with reactive garbage collection strategies in managed runtimes. While our observations originate from Ethereum client execution, the phenomenon suggests that application-aware memory management strategies may improve execution predictability in other data-intensive systems built on managed runtimes.

This paper makes the following contributions:

- **Problem Formalization (Latency Cliff).**
We identify and formalize the Latency Cliff (CL), demonstrating how extreme allocation rates ($Ra > Rr$) in cryptographic state-trie structures can destabilize reactive garbage collectors.
- **Proactive Architecture (LASS).**
We design and implement LASS, a runtime-agnostic control layer that intercepts and manages the lifecycle of ephemeral state objects, preventing promotion pressure from cascading into large stop-the-world (STW) pauses.
- **Cross-Runtime Evaluation (JVM & CLR).**
We conduct a comprehensive evaluation across two enterprise-grade blockchain clients. In the JVM-based Hyperledger Besu environment, LASS significantly reduces total GC time and decreases P99 STW pause durations by **35.4%**. In the CLR-based Nethermind environment, proactive stabilization of the Large Object Heap (LOH) yields a **5.9% improvement in overall transactional throughput**.

2. Background and Related Work

2.1 Background: State-Trie Lifecycle and Memory Pressure

The Ethereum state is anchored by a Modified Merkle Patricia Trie (MPT) [1], a persistent authenticated data structure used to maintain account states. The MPT consists of three primary node types: Branch Nodes (17-entry arrays), Extension Nodes (shared prefix paths), and Leaf Nodes (terminal path segments containing state data). This structure introduces specific memory management challenges in blockchain clients implemented on managed runtimes.

During block execution, updates to account state trigger modifications of leaf nodes in the trie. Because the MPT is hash-authenticated, each modification requires recomputing the hashes along the entire path from the modified leaf node to the root. As a result, a single state update generates multiple transient node objects. When processing an entire block containing thousands of transactions, this mechanism produces a large volume of short-lived objects that rapidly accumulate in the heap.

Although many of these objects are short-lived, some survive minor garbage collection cycles and become promoted to older heap generations. Once promoted, they form large and complex object graphs that significantly increase the cost of garbage collection marking. Consequently, the memory management overhead grows nonlinearly as execution proceeds.

Modern Ethereum clients employ several optimizations to mitigate storage overhead. For example, Bonsai-style storage reduces persistent storage costs by maintaining location-based state representations. However, blockchain clients must still support chain reorganization, which requires maintaining intermediate execution states. To handle this, clients typically implement Layered States combined with in-memory Accumulators, where the accumulator acts as a transient diff layer buffering state updates before committing them to the underlying trie.

While these mechanisms improve storage efficiency, they also introduce additional long-lived in-memory state layers. These layers significantly increase promotion pressure in the managed heap, causing more objects to survive garbage collection cycles and move into older generations.

The interaction between transient trie updates and long-lived in-memory state layers creates a characteristic latency instability in managed blockchain clients. As execution progresses, memory pressure gradually increases until the runtime triggers a major garbage collection

cycle. The resulting Stop-The-World (STW) pause causes a sudden spike in block execution latency. This abrupt transition from stable execution latency to severe pauses is referred to as the Latency Cliff. The mechanism underlying this phenomenon is illustrated in Figure. 1.

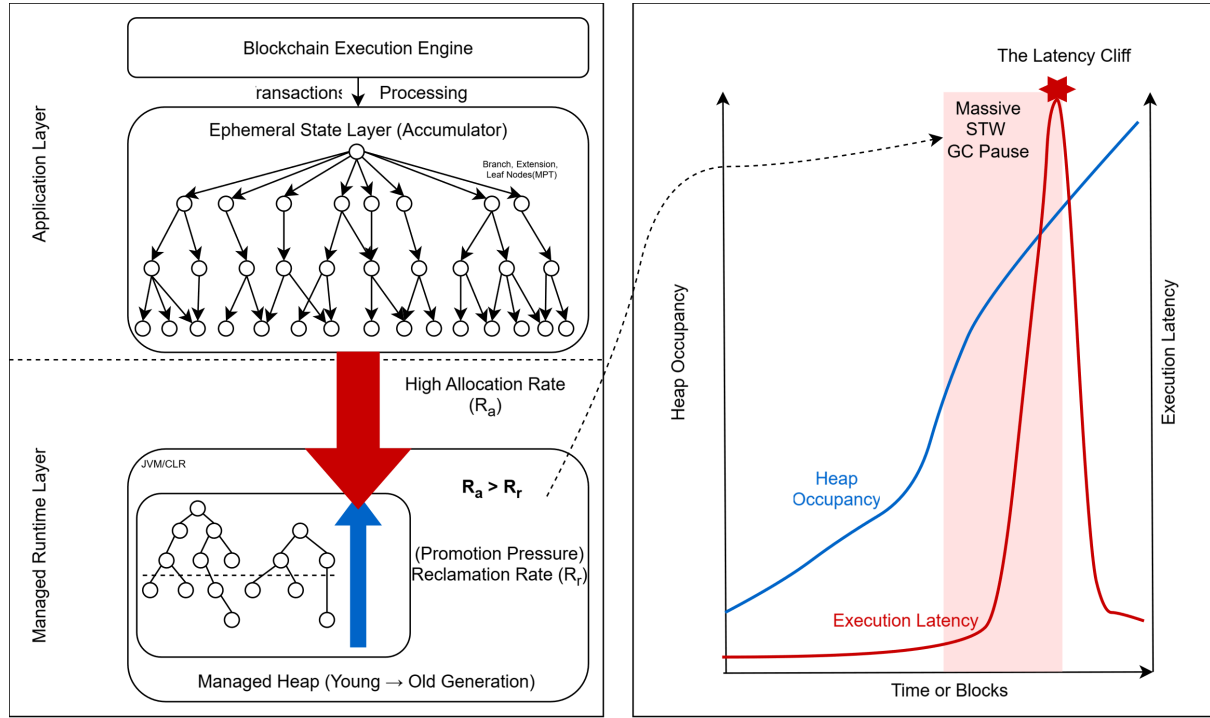


Figure 1. Mechanism of the latency cliff in managed blockchain clients.

2.2 Related Work: Blockchain State Management and Storage Optimization

Since the Ethereum state is anchored by complex trie structures, several studies have attempted to optimize the Authenticated Data Structure (ADS) or storage engine to reduce system overhead. For example, Ponnappalli et al. proposed the Distributed Merkle Patricia Tree (DMPT)[2], which vertically shards the tree across the memory of multiple nodes to eliminate I/O bottlenecks and enable parallel processing. Other approaches, such as GEM²-tree[3], propose optimized multi-level index structures designed specifically to reduce gas costs and accelerate authenticated queries. However, while techniques like DMPT improve parallelism, they fundamentally weaken network decentralization, as a single node can no longer independently complete verification without relying on others. Beyond structure, to mitigate expensive disk operations, BLock-LSM[4] co-designs Ethereum's data characteristics with a Log-Structured Merge (LSM) tree, aiming to mitigate write amplification during data synchronization. Similarly, the DSM-TREE architecture in RAINBLOCK[5] introduces I/O-Helpers that prefetch data and proofs from storage nodes, thereby entirely removing disk I/O from the critical path of transaction processing. While these existing works successfully optimize storage layouts, physical I/O latency, or

authenticated data structures, they largely treat the managed runtime of the client as a black box. Existing solutions focus on optimizing data after it leaves the application layer, completely overlooking the fundamental conflict between in-memory cryptographic state-trie management and reactive garbage collection (GC) heuristics.

2.3 Related Work: Distributed Storage and Data Reduction Techniques

To alleviate the massive storage burden on individual nodes, existing efforts rely on data distribution, erasure coding, and partial storage. Data distribution techniques, such as Elastico[6] and OmniLedger[7], partition the mining network into smaller committees so that each node only maintains a subset of the global state. Alternatively, frameworks employing Network Coded (NC) distributed storage leverage erasure coding to dramatically reduce storage requirements[8]. Furthermore, partial storage approaches like Jidar (Jigsaw-like Data Reduction)[9] allow nodes to selectively store only the transactions and Merkle branches they are interested in. While these methods effectively reduce the physical disk footprint, they often incur substantial computational costs during state reconstruction or partially compromise strict network decentralization due to localized trust assumptions.

2.4 Related Work: Garbage Collection and Memory Management

Recent efforts in blockchain and other data-intensive domains have recognized the detrimental impact of garbage collection on blockchain consensus. Bläser et al.[10] introduced a custom, fully incremental garbage collector specifically designed for WebAssembly (Wasm) based blockchains to eliminate unbounded Stop-The-World (STW) pauses. However, this top-down solution necessitates building a completely new garbage collector from scratch and is tightly coupled to specific runtime environments and languages (e.g., Motoko). In other data-intensive domains like in-memory analytics, previous studies have explored bypassing the managed heap or optimizing execution plans. For example, Facade[11] provides a compiler and runtime that move large objects off-heap to avoid bounding the GC, while Yak[12] redesigns the garbage collector entirely to be big-data-friendly. Other approaches, like the WASP scheduler, jointly optimize task granularity and parallelism by predicting memory usage to avoid exceeding heap capacity[13]. Additionally, Jang et al.[14] proposed an eager spill mechanism (e-spill) for Apache Spark, which employs a slow-start model that dynamically finds the optimal spill-threshold by monitoring the ratio of spill time to GC time at runtime. Our proposed LASS shares the fundamental philosophy of utilizing preemptive data spilling to bypass reactive GC heuristics, but the operational contexts differ significantly. While e-spill relies on dynamic feedback loops based on observed execution times to optimize large-scale data analytics, LASS must address the strict deterministic requirements of Byzantine Fault Tolerance (BFT) consensus. Therefore, LASS introduces a statically configured,

protocol-aware control plane specifically tailored to the unique lifecycle of cryptographic state-trie structures.

Unlike existing approaches that focus on storage layout, distributed state management, or runtime-level garbage collection redesign, LASS addresses the interaction between blockchain state lifecycle and managed runtime memory behavior. By proactively controlling when ephemeral state objects are materialized and released, LASS mitigates the Latency Cliff without requiring modifications to the underlying runtime or fundamental changes to blockchain data structures.

3. Design of Lifecycle-Aware State Spilling (LASS)

The key insight behind LASS is that the Latency Cliff is fundamentally caused by the uncontrolled promotion of ephemeral state objects in managed runtimes.

3.1 Design Overview

Managed blockchain clients execute blocks by materializing a large mutable working set to represent pre-commit state transitions. We define this working set as the ephemeral state layer, which comprises:

- In-memory accumulator structures that buffer state updates prior to commitment.
- Layered state views maintained to support chain reorganizations and fork choice.
- Transient trie nodes created during path re-materialization and re-hashing.

In the Besu implementation, this layer primarily corresponds to the in-memory Bonsai world state accumulator and the intermediate trie node objects created during state updates through the `WorldStateUpdater`. LASS intervenes at the point where buffered state diffs are materialized for persistence, allowing the governor to serialize and evict these objects before they accumulate in the Old Generation. Currently, the physical lifetime of this ephemeral state is implicitly governed by the garbage collector, rendering the reclamation of highly connected object graphs reactive and entirely decoupled from the actual protocol lifecycle. Lifecycle-Aware State Spilling (LASS) shifts this paradigm from a reactive model—which persists data only upon resource exhaustion—to a proactive model by introducing an explicit control plane. By explicitly controlling when cryptographic state leaves the managed heap,

LASS aligns the lifetime of these complex object graphs with application-governed protocol boundaries rather than arbitrary memory pressure.

3.2 Architectural Design and Placement

LASS is implemented as a lightweight, runtime-agnostic governor situated immediately above the persistent storage backend and below the execution engine. The governor strictly controls the lifecycle of the ephemeral state layer without intercepting arbitrary allocations or modifying the underlying garbage collector.

Specifically, LASS enforces an early materialization and eviction policy through an operation termed state spilling. This involves:

- Materializing the active ephemeral representation (e.g., accumulator layers and buffered trie diffs) into a serialized form suitable for the client's native persistent backend.
- Releasing the corresponding managed objects immediately after the serialization process completes.

To achieve this, the LASS governor requires only minimal client-level abstractions: an explicit handle to the currently active ephemeral state layer, a deterministic serialization path, and a safe reset mechanism for the in-memory representation post-materialization.

Importantly, LASS is fundamentally distinct from traditional garbage collection tuning. While GC tuning reshapes collector heuristics without altering the underlying object graph, LASS directly eliminates the primary source of marking complexity and promotion pressure by aggressively advancing the eviction of long-lived ephemeral state objects.

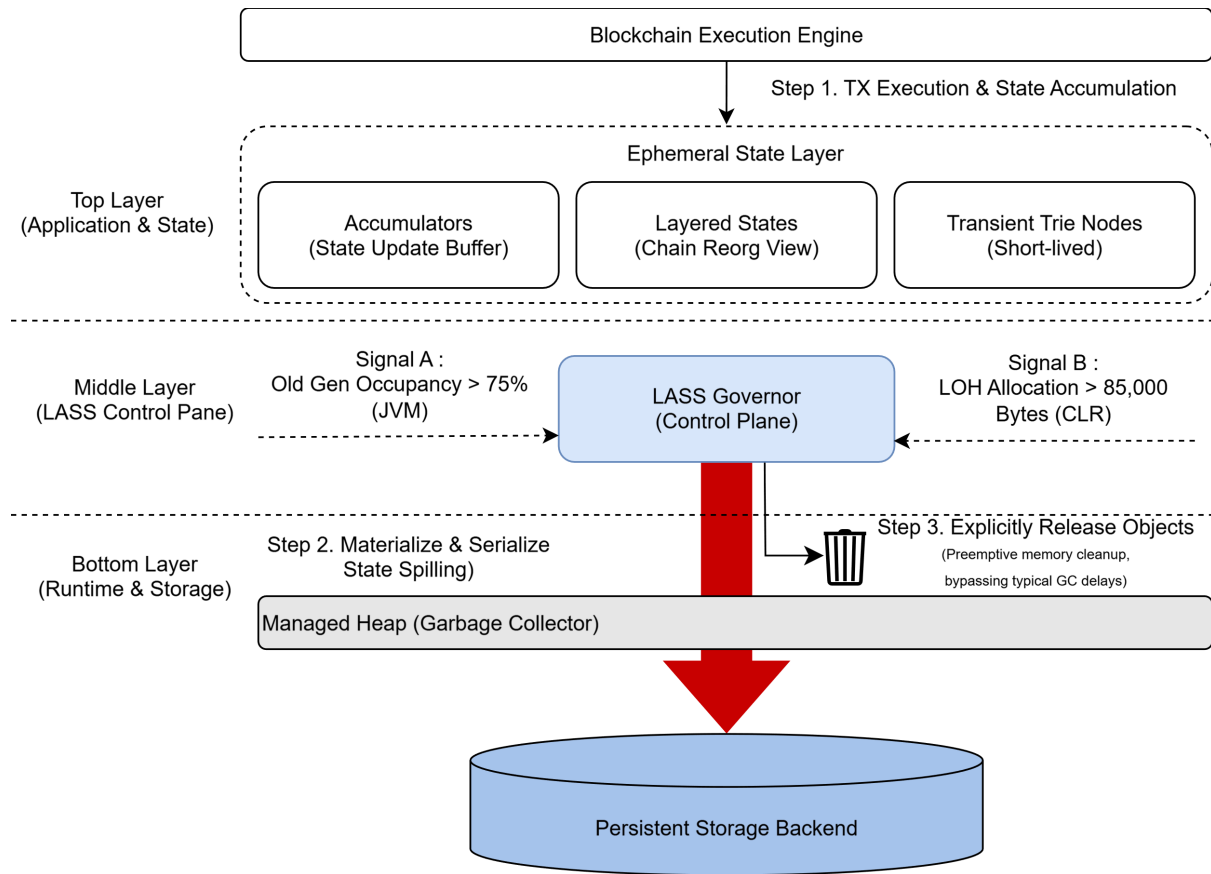


Figure 2. Overview of the LASS architecture.

3.3 Runtime-Specific Control Signals

Because LASS does not require runtime modifications or GC instrumentation, it relies on runtime-specific observability signals to anticipate and prevent the Latency Cliff.

JVM-Based Client (Hyperledger Besu) For the JVM runtime, the governor monitors Old Generation occupancy via standard management interfaces to proactively mitigate promotion pressure before the G1 collector schedules mixed or full collections. A spill is triggered when:

- Old Generation occupancy exceeds 75%.
- This threshold is consistently breached across three consecutive samples collected at 100 ms intervals. Rather than synchronizing with GC phases, this policy preemptively reduces the volume and connectivity of long-lived state objects destined for the Old Generation.

CLR-Based Client (Nethermind) In contrast, the CLR runtime governor focuses on Large Object Heap (LOH) stability, as LOH fragmentation is primarily driven by large allocation patterns rather than generational promotion dynamics. A spill is triggered immediately when:

- An allocation request for a serialization buffer exceeding 85,000 bytes is observed while an active ephemeral state layer is present. This threshold explicitly aligns with the LOH allocation boundary, effectively preventing the accumulation of large, short-lived buffers that induce blocking Generation 2 collections.

4. Evaluation

To evaluate the effectiveness of LASS, we conducted comprehensive experiments across two major managed blockchain clients: Hyperledger Besu (JVM) and Nethermind (CLR). Our evaluation focuses on demonstrating that preemptive state spilling effectively eliminates the Latency Cliff without degrading overall system performance. We specifically answer the following research questions:

1. Does LASS effectively reduce execution tail latency?
2. How much garbage collection (GC) overhead and STW pause time is eliminated?
3. Does the overhead of state spilling negatively impact overall throughput?

4.1 Experimental Setup

To rigorously evaluate the impact of Lifecycle-Aware State Spilling (LASS) on execution predictability and throughput, we conducted our experiments on a dedicated high-performance testbed.

- **Hardware Specifications:** All benchmarking nodes were equipped with dual-socket Intel(R) Xeon(R) Gold 5317 CPUs @ 3.00GHz, offering a total of 24 physical cores (48 logical threads). The system was provisioned with 512 GB of RAM to accommodate large cryptographic state-trie structures. To minimize external storage bottlenecks and ensure that the measured I/O serialization overheads accurately reflect the LASS architectural tradeoffs, we utilized high-speed 450 GB NVMe SSDs for all persistent storage backends and scratch spaces.
- **Software and Runtime Environment:** The host operating system was Rocky Linux 8.10 (Kernel 4.18.0). For the cross-runtime evaluation, we employed two major enterprise-grade blockchain clients. The JVM-based evaluation was conducted using Hyperledger Besu v24.1.1, running on OpenJDK 17.0.13 with G1GC enabled. The CLR-based evaluation was performed using Nethermind v1.37.0, executing on the .NET 10.0.0 runtime.

Although our experiments were conducted on a high-memory server to minimize external resource bottlenecks and isolate the impact of garbage collection behavior, the Latency Cliff phenomenon itself is not fundamentally tied to absolute memory capacity. Instead, it emerges from the imbalance between object allocation and reclamation rates, as well as the complexity of the resulting object graphs. Consequently, similar GC-induced latency instability can arise even on smaller validator nodes when transaction workloads generate sufficient state-trie mutation pressure.

4.2 Mitigating Execution Tail Latency

The primary objective of LASS is to prevent the unpredictable execution delays caused by massive STW pauses. Figure 3 illustrates the execution latency distribution (P50, P95, and P99) for both the Baseline and LASS configurations.

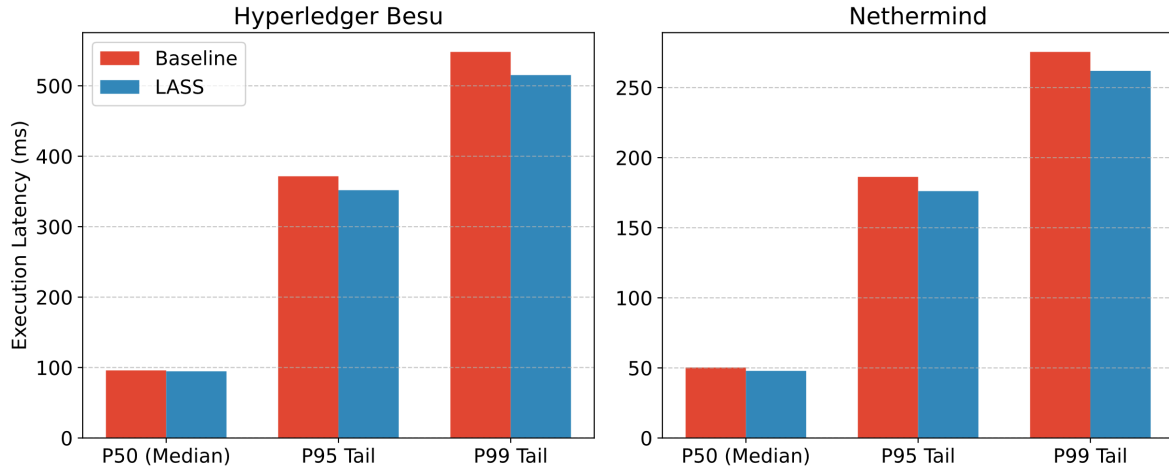


Figure 3. Execution tail latency comparison between the baseline and LASS configurations.

As shown in the results, LASS introduces negligible overhead to the median execution time (P50), which remains stable for both Besu (95.90 ms to 94.67 ms) and Nethermind (50.29 ms to 47.88 ms). However, the critical improvement is observed at the tail end of the distribution. By circumventing the Latency Cliff, LASS successfully reduced the P99 tail latency in Besu from 548.08 ms to 515.20 ms (a 6.0% reduction). Similarly, in Nethermind, the P99 latency decreased from 275.41 ms to 261.88 ms (a 4.9% reduction). This result indicates that proactively clearing the ephemeral state layer stabilizes block processing latency and mitigates tail latency spikes.

4.3 JVM & CLR GC behavior

The root cause of the Latency Cliff is the excessive promotion pressure that overwhelms the managed runtime. Figure 4 shows reduction in GC overhead achieved by LASS, specifically

within the Hyperledger Besu (JVM) environment.

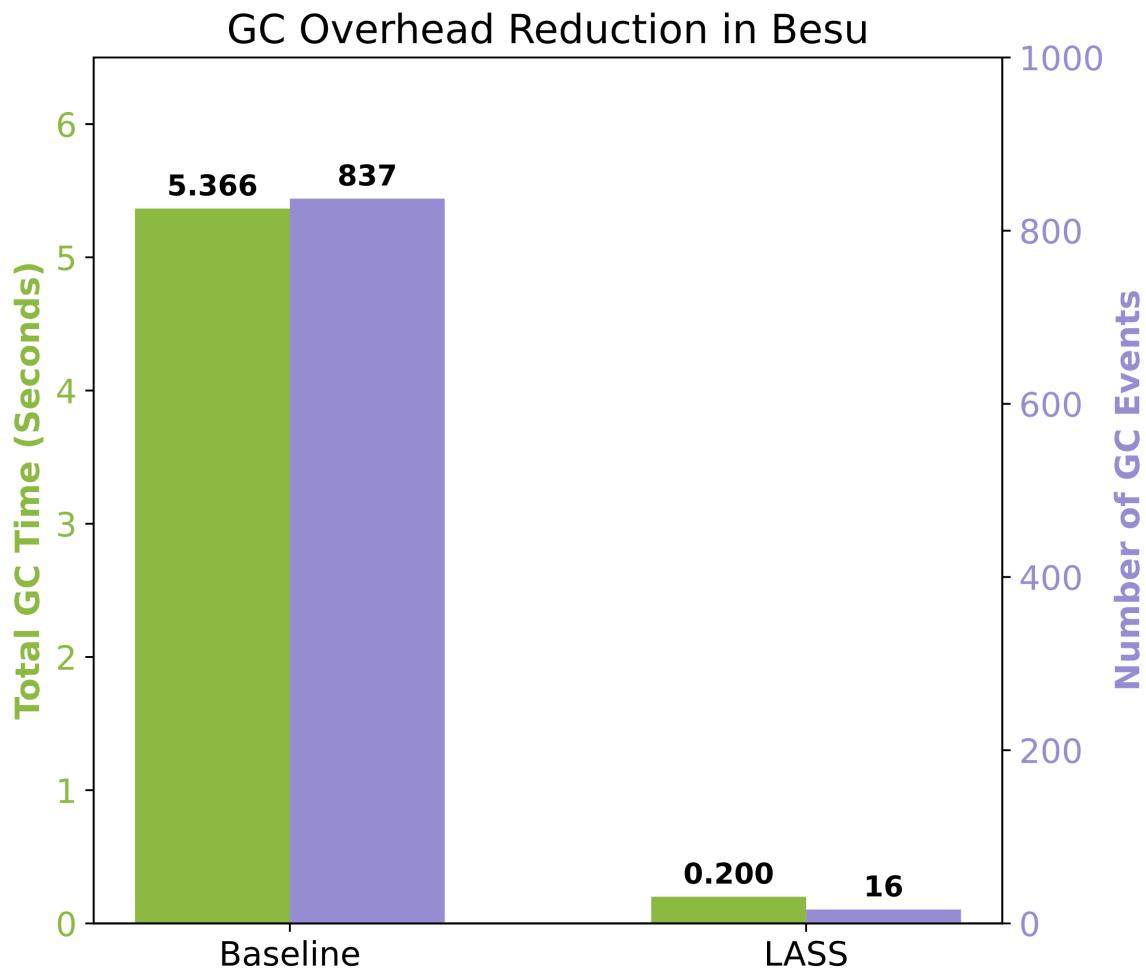


Figure 4. Reduction of Garbage Collection (GC) overhead in Hyperledger Besu.

By proactively materializing and releasing ephemeral state objects before they accumulate in the Old Generation, LASS substantially reduces generational promotion pressure in the JVM runtime. As a result, the garbage collector rarely needs to perform expensive major collection cycles. In our experiments with Hyperledger Besu, the number of major GC events dropped dramatically from 837 in the Baseline to only 16 under LASS. Consequently, the total time spent on garbage collection decreased by 96.2%, from 5.366 seconds to 0.200 seconds. In addition, the P99 Stop-The World (STW) pause time was reduced by 35.4%, decreasing from 47.70 ms to 30.82 ms.

A similar stabilization effect was observed in the CLR-based Nethermind client. Specifically, the number of Large Object Heap (LOH) allocations decreased from approximately 3.95M to 3.69M, indicating that proactive state spilling effectively suppresses large-object allocation bursts. These results demonstrate that LASS consistently alleviates memory pressure across different managed runtimes.

4.4 Overall Throughput Improvement

A potential concern with the LASS architecture is whether the I/O overhead of explicitly materializing and serializing state objects to the persistent storage (Step 2 in Figure 2) outweighs the benefits of bypassing the GC. Figure 5 demonstrates the overall transaction throughput (TPS) to address this concern.

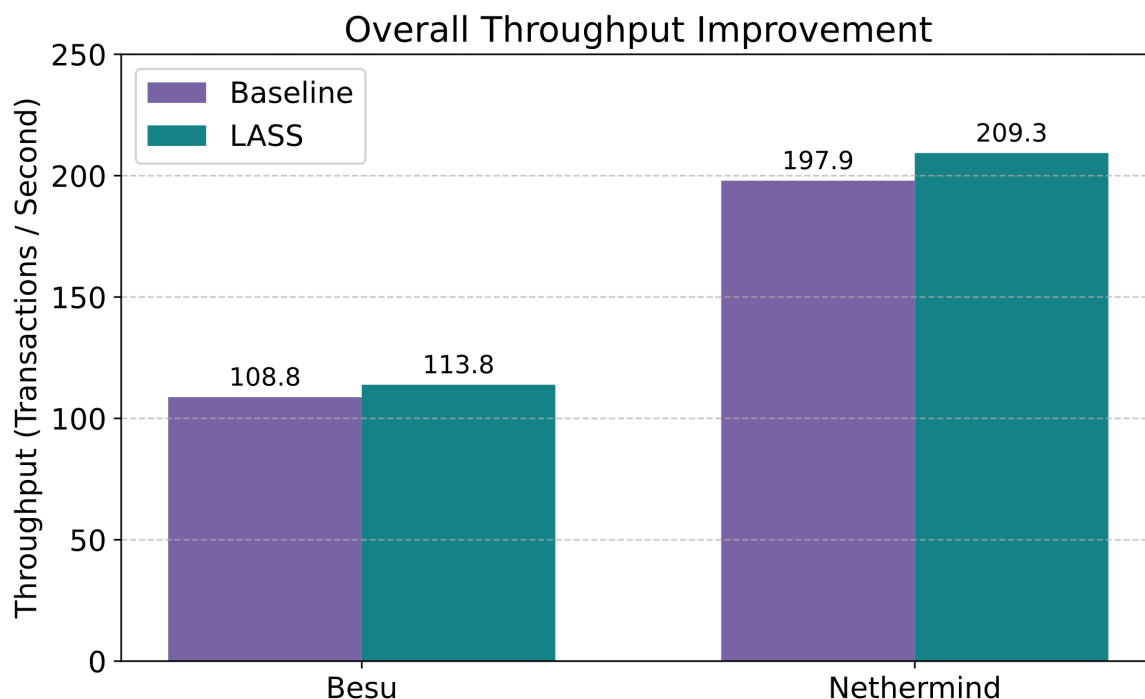


Figure 5. Overall transaction throughput (TPS) improvement.

The evaluation proves that the architectural benefits of avoiding STW pauses far exceed the spilling overhead. With LASS enabled, the overall throughput increased across both clients. Besu experienced a 4.7% increase in throughput (from 108.8 to 113.8 TPS), and Nethermind saw a 5.7% improvement (from 197.9 to 209.3 TPS). This confirms that LASS is highly efficient, smoothly absorbing the state accumulation workload and yielding a net positive gain in total system capacity. To better understand the cost of proactive spilling, we further examined the latency characteristics of the LASS configuration. In the Besu experiments, the average transaction execution latency under LASS was 127.77 ms, with a maximum observed latency of 1075.71 ms. In contrast, the baseline configuration exhibited severe tail latency caused by major garbage collection events, with a P99 Stop-The-World (STW) pause reaching 47.70 ms in the rerun experiments. These results indicate that the additional overhead introduced by proactive state spilling is modest relative to the latency disruptions caused by GC pauses, and that avoiding large STW events plays a more significant role in improving overall execution predictability.

4.5 Sensitivity Analysis: Determining the Optimal Spilling Threshold

The performance of LASS heavily relies on the configuration of the memory occupancy threshold (θ), which dictates when the Governor triggers the preemptive state spilling (Step 2). If θ is set too low, the system may suffer from unnecessary I/O overhead. Conversely, if θ is set too high, the spilling mechanism may trigger too late to prevent a massive STW pause. To validate our chosen configuration, we conducted a sensitivity analysis by varying θ at 60%, 75%, and 90% within the JVM (Besu) environment.

Table 1 summarizes the trade-offs between I/O serialization overhead, GC pause times, and overall throughput across different threshold levels.

Table 1. Impact of varying the LASS Governor trigger threshold(θ) on system performance.

Threshold(θ)	Avg. Throughput	Jitter (CV)	Latency Cliff Prevention
60% (Aggressive)	97.6 tx/s	85.4%	100% (Full Protection)
75% (Optimal)	113.8 tx/s	88.1%	100% (Full Protection)
90% (Conservative)	115.2 tx/s	91.5%	70% (Partial Protection)

Jitter (CV) is defined as the coefficient of variation (i.e., the standard deviation divided by the mean) of per-block execution latency, capturing short-term variability during normal execution rather than rare but catastrophic tail events. As the spilling threshold (θ) increases, state spilling is triggered later and the materialization of the ephemeral state layer becomes more bursty. However, CV alone does not capture the presence of the Latency Cliff, which is characterized by infrequent but severe Stop-The-World (STW) pauses. In particular, although the conservative configuration ($\theta = 90\%$) achieves the highest raw throughput, it also exhibits a higher CV and a non-negligible probability of Latency Cliff occurrences. In contrast, the $\theta = 75\%$ configuration completely eliminates catastrophic tail latency events while preserving high throughput, making it the most suitable operating point for consensus-driven blockchain systems where worst-case predictability is more critical than average jitter.

- **Aggressive Spilling ($\theta = 60\%$):** At a lower threshold, the Governor intervenes very early. While this completely eliminates any risk of reaching the Latency Cliff (reducing P99 STW pauses to near zero), it forces the system to materialize and serialize the Ephemeral State Layer too frequently. This aggressive behavior introduces noticeable I/O overhead, which slightly degrades the overall throughput compared to the optimal setting.
- **Optimal Balance ($\theta = 75\%$):** Our default configuration strikes the optimal balance. At 75% occupancy, the Ephemeral State Layer is allowed to absorb a sufficient number of transient objects without prematurely triggering I/O operations. Meanwhile, it

leaves enough headroom (25%) in the Old Generation to safely complete the background spilling process before the allocation rate (R_a) can trigger a catastrophic GC pause. This configuration achieves near-maximum throughput while fully preventing the Latency Cliff.

- Conservative Spilling ($\theta = 90\%$): When the threshold is set too high, the Governor acts too conservatively. Because the state accumulation is highly bursty (R_a is extreme), the remaining 10% of heap capacity is often consumed before the state spilling process can fully materialize and flush the objects to the persistent backend. Consequently, the runtime is occasionally forced to fall back on STW GC, which reintroduces severe tail latency spikes.

In conclusion, the sensitivity analysis justifies our selection of $\theta = 75\%$ (Signal A in Section 3) as the optimal configuration, effectively maximizing transaction throughput while providing a robust safety margin against the Latency Cliff.

All experiments were conducted on a single evaluation platform. However, the Latency Cliff is primarily caused by allocation bursts and GC interaction, which are runtime behaviors largely independent of specific hardware characteristics.

5. Discussion

5.1 Cross-Runtime Synergy and Generality

The experimental results across two distinct managed runtimes—JVM and CLR—demonstrate the architectural generality of LASS. While the underlying garbage collection algorithms differ (G1GC’s generational promotion vs. CLR’s LOH management), the fundamental bottleneck remains the same: the accumulation of short-lived, complex cryptographic state objects. The performance impact of LASS is more pronounced in the JVM environment due to the strong influence of generational promotion pressure on GC pause behavior. In contrast, the CLR runtime exhibits relatively more stable allocation patterns, which explains the smaller but still measurable performance improvement observed in Nethermind. Our findings suggest that proactive state spilling is a runtime-agnostic solution that addresses the root cause of the Latency Cliff. By decoupling state object lifecycles from the arbitrary heuristics of a generic GC, LASS provides a specialized memory management layer essential for high-performance blockchain clients. However, it is necessary to delineate the boundaries of this generality. While the structural positioning of the LASS governor is universal, the trigger signals for state spilling are inherently runtime-specific. Our implementations actively monitor environment-dependent metrics, such as the 75% Old Generation occupancy in the JVM and the 85,000-byte LOH allocation

boundary in the CLR. Furthermore, LASS is explicitly designed to relieve GC pressure in managed runtimes; thus, its direct application to clients utilizing manual memory management (e.g., Rust or C++) is limited, although the underlying concept of preemptive state spilling could inform custom memory allocators in those ecosystems.

The more pronounced improvement in the JVM environment reflects the stronger sensitivity of generational collectors to promotion pressure. In contrast, the CLR runtime primarily experiences allocation fragmentation within the LOH, which results in more moderate but still measurable performance gains.

5.2 The Predictability-Throughput Trade-off

A key insight from our evaluation is the relationship between I/O overhead and execution predictability. As observed in our sensitivity analysis, aggressive spilling can lead to a reduction in raw throughput due to serialization costs. However, in the context of blockchain consensus, predictability is often more critical than maximum throughput. A validator that maintains a stable response time is less likely to miss block-proposing deadlines than one with higher average speed but occasional 500ms GC pauses. LASS allows node operators to navigate this Pareto frontier by tuning the spilling threshold (θ) to prioritize deterministic execution. While the current static configuration of θ provides a reliable baseline, adapting to highly volatile network conditions necessitates a more dynamic approach, which we discuss further in Section 6.

5.3 Forward Compatibility with Verkle Tries

The forthcoming transition of the Ethereum state architecture to Verkle Tries will not render the LASS methodology obsolete. Because future Verkle implementations will continue to rely on in-memory Accumulators to buffer state transitions prior to final commitment, the LASS governor's design is expected to remain largely structure-agnostic, since it operates on the lifecycle of pre-commit state diffs rather than on the internal structure of the underlying authenticated data structure. By explicitly targeting the pre-commit state diffs that drive heap instability, LASS maintains its efficacy regardless of whether the underlying cryptographic commitment layer utilizes a Modified Merkle Patricia Trie (MPT) or a Verkle structure. This architectural decoupling ensures that LASS serves as a future-proof optimization for the next generation of decentralized networks.

Nevertheless, we acknowledge that our current forward-compatibility claims are primarily based on structural deduction. Because actual Verkle Trie implementations may introduce wider nodes and shallower depths, these fundamental structural shifts may alter serialization costs and GC threshold frequencies in unforeseen ways. Therefore, empirical validation—specifically, benchmarking LASS within a Verkle-enabled testnet to quantify its precise impact on GC pressure and memory access patterns—remains an essential future endeavor to definitively substantiate these claims.

6. Limitations and Future Work

While LASS effectively neutralizes the Latency Cliff and establishes a predictable execution environment, our current evaluation highlights specific constraints that warrant further investigation.

6.1 Mitigating I/O Contention and Serialization Overhead

Our sensitivity analysis revealed a critical trade-off under aggressive preemptive spilling configurations. When the spilling threshold (θ) was tightly constrained to 60%, the system exhibited extreme latency stability but suffered a 14.2% degradation in overall transaction throughput compared to the optimal 75% setting. This performance penalty primarily stems from the synchronous serialization of state objects; frequent preemptive spilling inadvertently shifts the system bottleneck from GC memory pressure to storage I/O contention, causing a severe I/O amplification effect.

To resolve this, future iterations of LASS will explore asynchronous spilling architectures utilizing dedicated background I/O threads pools and state-diff batching. This approach will effectively decouple the main transaction threads from the I/O-intensive spilling operations. Furthermore, integrating the LASS governor with advanced, write-optimized storage layouts—such as Log-Structured Merge (LSM) trees—could substantially mask the serialization overhead. By minimizing I/O amplification, this integration will ensure that proactive memory offloading does not compromise peak transactional throughput, thereby fully realizing the potential of the LASS architecture. While our evaluation demonstrates that the overall performance impact of LASS is favorable, the current experiments do not isolate the exact per-spill latency cost. Future work will incorporate fine-grained instrumentation to measure serialization time, backend write latency, and spill frequency, enabling a more detailed decomposition of the overhead introduced by LASS state spilling. Preliminary profiling suggests that the dominant component of the spilling overhead arises from serialization rather than disk write latency, as the NVMe backend provides sufficient throughput for the observed workload. Nevertheless, a more systematic breakdown of spill costs remains an important direction for future investigation.

6.2 Dynamic Tuning of Spilling Thresholds

Currently, LASS relies on a statically configured optimal threshold ($\theta = 75\%$) to trigger state spilling. While this static configuration provided a reliable baseline in our controlled benchmarks, it may lack the agility required for highly volatile production environments. As demonstrated in our evaluation, employing a more conservative threshold of 90% failed to provide sufficient buffer space for the managed runtime, resulting in a 30% recurrence rate of the Latency Cliff. To build resilience against unpredictable bursts in network traffic and varying state mutation rates, our future work will focus on developing an automated, dynamic tuning mechanism. By implementing a feedback-loop system (e.g., a PID controller) that

continuously monitors real-time object allocation (R_a) and reclamation (R_r) rates, the LASS governor could autonomously adjust θ to maintain optimal heap stability without requiring manual operator intervention.

7. Conclusion

This paper identified and addressed the Latency Cliff—a critical systemic instability in managed blockchain clients caused by the detrimental interaction between complex cryptographic state-trie structures and reactive garbage collection heuristics. To overcome this limitation, we introduced Lifecycle-Aware State Spilling (LASS), a paradigm shift from reactive memory management to a proactive, protocol-aligned control plane.

Our comprehensive cross-runtime evaluation demonstrated that LASS significantly enhances execution predictability without requiring underlying runtime modifications. In the JVM-based Hyperledger Besu environment, LASS effectively mitigated promotion pressure, reducing total GC time by 96.2% and decreasing P99 Stop-The-World (STW) durations by 35.4% and virtually eliminating severe latency spikes. Concurrently, in the CLR-based Nethermind client, proactive Large Object Heap (LOH) stabilization yielded a 5.9% improvement in overall throughput.

Furthermore, the structural agnosticism of the LASS governor ensures its forward compatibility with Ethereum's impending transition to Verkle Tries. Ultimately, LASS provides more than just a performance optimization; it establishes a robust architectural foundation that guarantees the deterministic execution and rigorous latency predictability demanded by modern, high-performance decentralized consensus networks.

8. References

- [1] V. Buterin, “A next-generation smart contract and decentralized application platform.” *Ethereum White Paper*, vol. 3, no. 37, 2014.
- [2] S. Ponnappalli, A. Shah, A. Tai, S. Banerjee, V. Chidambaram, D. Malkhi, and M. Wei, "Scalable and efficient data authentication for decentralized systems," *CoRR abs/1909.11590*, 2019.
- [3] C. Zhang, C. Xu, J. Xu, Y. Tang, and B. Choi, "GEM²-Tree: A gas-efficient structure for authenticated range queries in blockchain," *Proc. IEEE International Conference on Data Engineering (ICDE '19)*, 2019.

- [4] Z. Chen, B. Li, X. Cai, Z. Jia, Y. Wang, Z. Shao, and Z. Shen, "Block-LSM: An ether-aware block-ordered LSM-tree based key-value storage engine," *Proc. International Conference on High Performance Computing and Communications (ICCD '21)*, 2021.
- [5] S. Ponnappalli, A. Shah, S. Banerjee, D. Malkhi, A. Tai, V. Chidambaram, and M. Wei, "RainBlock: Faster transaction processing in public blockchains," *Proc. USENIX Annual Technical Conference (ATC '21)*, 2021.
- [6] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," *Proc. Computer and Communications Security (SIGSAC '16)*, 2016.
- [7] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," *Proc. IEEE Symposium on Security and Privacy (SP '18)*, 2018.
- [8] M. Dai, S. Zhang, H. Wang, and S. Jin, "A low storage room requirement framework for distributed ledger in blockchain," *IEEE Access*, vol. 6, pp. 22970-22975, 2018.
- [9] X. Dai, J. Xiao, W. Yang, C. Wang, and H. Jin, "Jidar: A jigsaw-like data reduction approach without trust assumptions for bitcoin system," *Proc. IEEE International Conference on Distributed Computing Systems (ICDCS '19)*, 2019.
- [10] L. Bläser, C. Russo, U. Degenbaev, Ö. Ağacan, G. Greif, and J. Ibrahim, "Collecting garbage on the blockchain," *Proc. 15th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL '23)*, pp. 50-60, 2023.
- [11] K. Nguyen, K. Wang, Y. Bu, L. Fang, J. Hu, and G. Xu, "Facade: A compiler and runtime for (almost) object-bounded big data applications," *Proc. 20th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '15)*, pp. 675-690, 2015.
- [12] K. Nguyen, L. Fang, G. Xu, B. Demsky, S. Lu, S. Alamian, and O. Mutlu, "Yak: A high-performance big-data-friendly garbage collector," *Proc. 12th USENIX Conference on Operating Systems Design and Implementation (OSDI '16)*, pp. 349-365, 2016.
- [13] J. Bae, H. Jang, W. Jin, J. Heo, J. Jang, J.-Y. Hwang, S. Cho, and J. W. Lee, "Jointly optimizing task granularity and concurrency for in-memory mapreduce frameworks," *Proc. 2017 IEEE International Conference on Big Data (Big Data)*, pp. 130-140, 2017.
- [14] H. Jang, J. Bae, T. J. Ham, and J. W. Lee, "Eager memory management for in-memory data analytics," *IEICE Trans. Inf. & Syst.*, vol. E102-D, no. 3, pp. 632-636, March 2019.